

Lisha Jin

AI/ML Software Tools & Platforms Course

6/26/2026

Professor: Dr. Chale

Final Project

For this final project, I wanted to build a ML model that can use a customer's profile to predict whether they would reply to an offer in a promotion campaign. This felt important, because marketing departments at a company can struggle to know which customers to target in campaigns and which ones are more likely to ignore these. Knowing which customers are more likely to respond to a campaign can make employees' jobs easier.

I downloaded a marketing_campaign.csv file from Kaggle. This dataset contained variables such as "ID", "Year_Birth", "Education", "Marital_Status", "Income", "Kidhome" (number of children in a customer's household), "Teenhome" (number of teenagers the customer had in their household), "Dt_Customer" (date in which the customer enrolled with the company), "MntWines" (amount the customer spent on wine in the past 2 years), "MntFruits" (amount the customer spent on fruits in the past 2 years), "Complain" (did the customer complain in the last 2 years?), and Response ("did the customer accept the offer in a recent campaign?").

This dataset seemed like a good pick for my research question, because it contained a bunch of different variables that can be used to predict a customer's behavior. This dataset also had a mix of numerical and categorical variables. However, one weakness was the fact that there were missing values. Another area of concern was that it wasn't clear if some customers included the count of teenagers in their household in the count of children in their home.

The dataset was imported by using the `read_csv()` function. The “sep=’\t’” function had to be used on the raw dataset before we could use other functions. Some variables were removed to narrow down the list of variables that could be used to make predictions for the target variable. The target variable (y) was the Response variable. Categorical features were prepared for preprocessing by using the one-hot encoding method.

While there were better ways to handle missing values in this dataset, I decided to remove rows with missing values entirely. This was done to make things simpler, but there were better approaches. Potential duplicate rows were addressed by using a function to identify and remove them. Birth years before 1926 were removed from the dataset because these might have been strong outliers.

Predictor variables included birth year, education, marital status, income, number of kids at a customer’s home, number of teenagers in their home, the amount of products purchased, number of online purchases, number of discount purchases, number of catalog purchases, number of store purchases, whether the customer complained, and the number of web visits from the client. 80% of the data was split into a training and validation dataset. The remaining 20% was used as a test set. `Random_state=42` was used to make the splits reproducible. The validation set was used to assess the model performance before making predictions for the test set.

The Response variable was used as the target one. Some new features were designed based on the existing variables to attempt to improve the predictions. The Age variable was created by subtracting date of birth from 2026. To make things easier, the total spending and purchases for different products were combined into one feature. The amount of kids and teenagers in a customer’s household was added to create a new variable.

A bar chart was used to better visualize the class imbalance for the Response variable from the dataset. A boxplot was used to see if there was a noticeable difference between customers with different income brackets and whether they responded to a campaign. A heatmap was used to visualize relationships between numerical features. This was helpful in getting a better idea of which variables were strongly or weakly correlated with one another.

A logistic regression model was used to classify customers based on predictor values into whether they were more likely to respond or not respond to a campaign. A pipeline was utilized to simplify preprocessing and model training steps by combining them into one workflow. OneHotEncoder was used on categorical variables to prepare them for the logistic regression model. The model was trained on the training dataset and made predictions for the validation and test datasets. Model performance was evaluated by looking at the metrics such as the F1-score on the classification report. A confusion matrix was also used to further assess the performance.

The model scored a weighted average f1-score of 0.83 which looks pretty good. However it only scored a 0.19 f1-score for the 1 class of the Response variable. This was partly due to the strong class imbalance. The model was way better at predicting if a customer would not respond to a campaign than if they would. The top 3 features that predicted if a customer would respond to a campaign was if they were in a relationship, if they were married or if they had a PhD. Customers that were in relationships or married were very unlikely to respond to a campaign based on this model's predictions.

There were some limitations to note. Because rows with missing values were removed, the amount of data that was used by the model was decreased. These rows could've contained valuable information that could have improved the model performance significantly. Logistic regression might have not been the best model to use for this research project since the

relationship between variables and the log-odds of whether a customer responds to a campaign is probably not linear. The class imbalance of the response variable could have been addressed with weighting strategies to improve the model.

Potential ideas for next steps include using weighting strategies to address class imbalance, using hyperparameter tuning, and using a model that's better fitted for this type of research. Pandas was used for loading, preparing and cleaning the dataset. NumPy was used for numerical functions. Matplotlib and Seaborn were used to make data visualizations for exploratory data analysis. Scikit-learn was used to preprocess data, one-hot-encoding, splitting the data into different sets, and training and evaluating the model.